

1 Zero to Hero Study Handbook: Multi-Agent AI Studio

This handbook is built from static analysis of the repository at `/home/ahmad/AI/multi-agent-ai-studio`. It uses real modules, classes, functions, routes, and config keys from this codebase.

1.1 Module 1: Foundations & Architecture

1.1.1 1.1 What this project does

Multi-Agent AI Studio is a local-first AI engineering platform with:

- a FastAPI backend (`apps/api/src/ai_studio`),
- a Next.js frontend (`apps/web`),
- LangGraph-inspired staged orchestration (`SupervisorRuntime`),
- local model runtime via Ollama (`OllamaClient`),
- PostgreSQL for core state,
- Qdrant for vector retrieval,
- Redis for queue/rate-limit state,
- ARQ worker for async run execution,
- OpenTelemetry tracing hooks.

Main product use cases in current code:

1. Authenticated chat-based multi-stage agent execution (`/api/v1/chat`).
2. Agent/workflow CRUD and workflow validation (`/api/v1/agents`, `/api/v1/workflows`).
3. Memory lifecycle operations (`/api/v1/memory`).
4. RAG document ingestion and retrieval (`/api/v1/rag/*`).
5. Tool execution with confirmation and audit logging (`/api/v1/tools/*`).
6. Evaluation, experiments, logs, traces, settings, and system metrics APIs.

1.1.2 1.2 Core paradigms and patterns used

1. Modular monolith architecture. The backend is one deployable service, split by domain folders (`api`, `services`, `models`, `db`, `tools`).
2. Async I/O and dependency injection. FastAPI route handlers are async and use `Depends(...)` for auth, DB session, rate limit, and RBAC (`apps/api/src/ai_studio/api/deps.py`).
3. Typed contract-first API. Pydantic schemas in `apps/api/src/ai_studio/schemas/*.py` define request/response shapes.
4. Graph/stage orchestration pattern. `SupervisorRuntime` defines planner, decomposer, router, specialist, reviewer, critic stages in `apps/api/src/ai_studio/services/agent_runtime.py`.
5. Capability-based model routing. `ModelRouter.pick(task)` selects models by inferred capabilities and score rather than hardcoded fixed model IDs (`apps/api/src/ai_studio/services/model.py`).
6. Queue-based background processing. `/runs` enqueues jobs to ARQ Redis queue; worker executes `execute_agent_run` (`workers/arq/src/ai_studio_worker/main.py`).
7. Safety-gated tool execution. Destructive operations require role + X-Confirm-Token and are audited (`apps/api/src/ai_studio/api/routers/tools.py`, `apps/api/src/ai_studio/api/deps.py`).
8. Persistence + observability split. Business state in PostgreSQL tables (`models/entities.py`), vector state in Qdrant, telemetry via `system_metrics` table and OTEL instrumentation.

1.1.3 1.3 High-level architecture and interactions

```
[Next.js Frontend]
| (Bearer JWT, REST/SSE)
v
[FastAPI API: ai_studio.main]
|-- Auth/RBAC/RateLimit (api/deps.py)
|-- Routers (api/routers/*.py)
|-- AppState (state.py)
|   |-- OllamaClient ---> [Ollama]
|   |-- ModelRouter
|   |-- SupervisorRuntime (planner->decomposer->router->specialist->reviewer->critic)
|   |-- RagService ---> [Qdrant]
|   |-- Job Queue ---> [Redis]
|   |-- Metrics loop ---> [PostgreSQL system_metrics]
|
|-- SQLAlchemy Session ---> [PostgreSQL]
|-- Tool execution ----> [Filesystem/Terminal/Python sandbox]

[ARQ Worker: ai_studio_worker.main]
|-- dequeues execute_agent_run from Redis
|-- uses Ollama + ModelRouter + SupervisorRuntime
|-- persists run output + TraceRecord to PostgreSQL
```

1.1.4 1.4 Important architectural realities in current code

1. SupervisorRuntime builds a LangGraph object (self._graph) but runtime execution currently uses an explicit ordered stage loop in run_with_trace and stream_events.
2. Task routing is keyword-based in _route_task_type, not classifier-based.
3. OCR/Vision/Browser workspace pages currently send text prompts into chat flows; there is no dedicated browser automation or image-upload execution pipeline in those pages.
4. RAG is fully implemented for file ingestion, GitHub/web connector ingestion, and hybrid retrieval with reranking.

1.2 Module 2: Repository Map

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
pyproject.toml	Python deps, tooling, test/lint config	N/A	requires-python, dependencies, [tool.pytest], [tool.mypy], [tool.ruff]
Makefile	Common developer commands	setup, run-api, run-worker, test, lint, compose-up	UV_CACHE_DIR, .env.example
Environment contract	N/A	APP_*, POSTGRES_*, REDIS_URL, QDRANT_URL, OLLAMA_*, TOOL_*	
docker-compose.yml	Full stack service topology	N/A	service defs for api, worker, ollama, web, postgres, redis, qdrant, minio, otel-collector
infra/compose/api.Dockerfile	API container build/start	N/A	runs alembic upgrade head then uvicorn
infra/compose/worker.Dockerfile	Worker container build/start	N/A	runs uv run arq ai_studio_worker.main.WorkerSettings
apps/api/src/ai_studio/main.py	FastAPI app entrypoint and lifespan	lifespan, app, root	CORS via settings.allowed_origins
apps/api/src/ai_studio/state.py	Global runtime state initialization	AppState, build_app_state, _collect_system_metrics_loop	metrics_sample_interval_seconds, tool manifests
apps/api/src/ai_studio/core/config.py	Central settings model	Settings, database_url, bootstrap_models	env-mapped fields (app/db/ollama/tool/otel/github)
apps/api/src/ai_studio/api/router.py	API v1 router composition	api_router	prefix /api/v1
apps/api/src/ai_studio/api/deps.py	Auth,		

RBAC, rate limit, audit helpers | `get_current_user`, `require_roles`, `rate_limit`, `append_audit_log`, `require_confirmation_token` | `RATE_LIMIT_RPM`, `APP_ENV` || `apps/api/src/ai_studio/models/entities.py` | SQLAlchemy entity model definitions | `User`, `Agent`, `Workflow`, `Chat`, `MemoryRecord`, `Document`, `AgentRun`, `TraceRecord`, etc. | `enums` `UserRole`, `RunStatus` || `apps/api/alembic/versions/20260627_0001_i` | Initial DB schema migration | `upgrade`, `downgrade` | `table/index` creation || `apps/api/alembic/versions/2026` | System metrics table migration | `upgrade`, `downgrade` | `system_metrics` table || `apps/api/src/ai_studio/ser` | Multi-stage supervisor runtime | `SupervisorRuntime`, `_planner`, `_decomposer`, `_router`, `_specialist`, `_reviewer`, `_critic`, `run_with_trace`, `stream_events` | `AgentState`, `TaskType` mapping || `apps/api/src/ai_studio/services/model_router.py` | Dynamic model discovery and routing | `ModelRouter.refresh`, `pick`, `snapshot`, `set_custom_rule` | `model_router_refresh_seconds`, preferred model settings || `apps/api/src/ai_studio/services/ollama_client.py` | Ollama HTTP integration | `health`, `list_models`, `ensure_models`, `embeddings`, `generate`, `stream_generate` | `ollama_base_url`, `ollama_disable_thinking`, `timeouts` || `apps/api/src/ai_studio/services/rag_service` | RAG ingestion/retrieval pipeline | `_extract_text`, `_chunk_text`, `ingest_local_file`, `ingest_connector`, `retrieve` | Qdrant collection `_COLLECTION_NAME=studio_chunks` || `apps/api/src/ai_studio/services/work` | WorkflowSpec validation/compilation | `WorkflowCompiler.compile`, `CompiledWorkflow` | `node/edge` validity rules || `apps/api/src/ai_studio/services/memory_service.py` | Memory persistence lifecycle | `create`, `list`, `update`, `forget`, `prune_expired` | `memory_type`, `scope`, `salience`, `ttl_days` || `apps/api/src/ai_studio/services/cost_estimator.py` | Cost estimation model | `CostEstimator.estimate` | `token/GPU/CPU` rates || `apps/api/src/ai_studio/services/system_metrics` | Host telemetry snapshotting | `SystemMetricsService.snapshot` | `CPU/RAM/GPU` values || `apps/api/src/ai_studio/tools/filesystem.py` | Filesystem tool implementation | `read_text`, `write_text`, `move_path`, `copy_path`, `delete_path`, `search_text` | `AI_STUDIO_ALLOWED_ROOT` || `apps/api/src/ai_studio/tools/terminal.py` | Terminal tool implementation | `run_command`, `_run_container_command`, `_run_host_command` | `tool_execution_mode`, `tool_container_*` || `apps/api/src/ai_studio/tools/python_exec.py` | Python execution tool implementation | `run_python` | container fallback behavior || `apps/api/src/ai_studio/api/routers/chat.py` | Chat + SSE orchestration API | `create_message`, `stream_chat` | `token_usage`, `TraceRecord` metadata || `apps/api/src/ai_studio/api/routers/rag.py` | RAG document/connectors/retrieval API | `create_document`, `ingest_connector`, `ingest_document_file`, `retrieve` | upload dir resolution via `AI_STUDIO_UPLOAD_DIR` || `apps/api/src/ai_studio/api/routers/tools.py` | Tool list and execution endpoints | `execute_tool`, `fs_*`, `terminal_exec`, `python_run` | role gating, confirmation token, arg schema validation || `workers/arq/src/ai_studio_worker/main.py` | Background run worker entrypoint | `execute_agent_run`, `WorkerSettings` | Redis DSN from settings || `apps/web/lib/api.ts` | Frontend API client/auth token handling | `apiRequest`, `refreshAccessToken`, `ensureSession` | `NEXT_PUBLIC_API_BASE_URL`, `localStorage` keys || `apps/web/components/chat/chat-panel.t` | Streaming chat UI (SSE event parsing) | `send` | handles `event:stage`, `event:error`, `event:end` || `apps/web/components/workflow/editor.tsx` | Visual workflow editor | `WorkflowEditor`, `validateWorkflow`, `saveWorkflow` | maps to `/workflows` + `/workflows/validate` || `apps/web/app/*/page.ts` | Product pages by feature area | page components | each page maps to corresponding API routes || `scripts/bootstrap.sh` | Local bootstrap script | N/A | `uv venv`, `uv sync`, `alembic migrate` || `scripts/e2e_live_run.sh` | Full-stack live E2E script (reference) | `api_call`, `assert_jq` + many endpoint calls | `BASE_URL`, `CONFIRM_TOKEN`, artifact paths |

1.3 Module 3: Core Execution Flows

1.3.1 3.1 Flow A: API startup and runtime state boot

Entry path:

1. `uvicorn ai_studio.main:app --app-dir apps/api/src ...`
2. lifespan in `apps/api/src/ai_studio/main.py`.

Step-by-step:

1. `configure_logging()` sets JSON logs (`apps/api/src/ai_studio/core/logging.py`).
2. `ensure_schema_current(engine)` verifies DB schema head revision matches Alembic (`apps/api/src/ai_studio/db/migrations.py`).
3. DB connectivity check executes `SELECT 1`.
4. `build_app_state()` initializes Ollama client, model router, supervisor runtime, rag service, tool registry, Redis job queue, and starts metrics loop.
5. `set_app_state(state)` stores singleton state for route handlers.
6. On shutdown, `shutdown_app_state()` closes metrics task, Ollama HTTP client, RAG HTTP client, and Redis pool.

Minimal code fragment:

```
state = await build_app_state()
set_app_state(state)
```

Input/Output shape:

- Input: environment variables from `.env`.
- Output: initialized `AppState` object with fields: `ollama_client`, `model_router`, `supervisor_runtime`, `rag_service`, `tool_registry`, `system_metrics`, `job_queue`, `metrics_task`.

1.3.2 3.2 Flow B: Authentication and session lifecycle

Relevant files:

- `apps/api/src/ai_studio/api/routers/auth.py`
- `apps/api/src/ai_studio/core/security.py`
- `apps/web/lib/api.ts`

Step-by-step register/login:

1. `POST /api/v1/auth/register` validates `RegisterRequest`.
2. Password hashed via `hash_password` (Argon2).
3. `_issue_token_pair` creates access + refresh JWTs.
4. `_store_refresh_token` stores refresh token JTI in Redis with expiry.
5. Frontend stores tokens in `localStorage` keys `studio_access_token`, `studio_refresh_token`.

Refresh flow:

1. `POST /api/v1/auth/refresh` with `refresh_token`.
2. `_assert_refresh_token_active` decodes token and checks JTI presence in Redis.
3. Old JTI deleted, new token pair issued.
4. Frontend `apiRequest` retries once on 401 by calling `refreshAccessToken`.

Request/response shapes:

```
POST /api/v1/auth/login
{
  "email": "user@example.com",
  "password": "StrongPass123!"
}
```

```

200 OK
{
  "access_token": "...",
  "refresh_token": "...",
  "token_type": "bearer"
}

```

1.3.3 3.3 Flow C: Synchronous chat orchestration (POST /chat/{id}/messages)

Relevant file:

- apps/api/src/ai_studio/api/routers/chat.py

Step-by-step:

1. Validate chat ownership.
2. Create AgentRun(status=queued) and user ChatMessage(role="user").
3. Mark run running.
4. Call runtime.run_with_trace(payload.content).
5. Persist assistant message with token_usage fields.
6. Persist TraceRecord with timeline metadata.
7. Mark run completed and save output payload.

runtime.run_with_trace stage order:

1. planner
2. decomposer
3. router
4. specialist
5. reviewer
6. critic

Request shape (ChatMessageCreate):

```

{
  "content": "SQL Agent Task: Find top workflows by run volume",
  "model": null,
  "agent_id": null,
  "context": {"workspace": "sql"}
}

```

Response shape (ChatMessageRead):

```

{
  "id": "...",
  "chat_id": "...",
  "role": "assistant",
  "content": "...final response...",
  "citations": [],
  "token_usage": {
    "prompt_tokens": 42,
    "completion_tokens": 180,
    "latency_ms": 812.3,

```

```

    "metadata": {"trace_id": "...", "timeline": [...]},
    "critique": "...",
    "run_id": "...",
    "trace_id": "..."
  },
  "created_at": "...",
  "updated_at": "..."
}

```

1.3.4 3.4 Flow D: Streaming chat orchestration (GET /chat/{id}/stream)

Relevant files:

- backend SSE producer: `apps/api/src/ai_studio/api/routers/chat.py`
- frontend SSE consumer: `apps/web/components/chat/chat-panel.tsx`

Step-by-step:

1. Backend creates queued run and user message.
2. Emits event:meta with run ID.
3. Iterates `runtime.stream_events(prompt)`.
4. Emits event:stage frames for stage start/complete.
5. On final payload, chunks response text and sends data: frames.
6. Persists run/message/trace, emits event:end.

Frontend parser behavior:

1. Reads stream with `ReadableStream.getReader()`.
2. Splits frames by `\n\n`.
3. Parses event:stage JSON into timeline UI.
4. Appends text chunks into last assistant message.
5. Stops on event:end.

SSE event examples:

```

event:meta
data:{"run_id":"..."}

```

```

event:stage
data:{"event":"stage","status":"complete","stage":"planner","latency_ms":123.4,"model":"qwer

```

```

event:end
data:done

```

1.3.5 3.5 Flow E: Model routing and capability selection

Relevant file:

- `apps/api/src/ai_studio/services/model_router.py`

How it works:

1. `refresh()` fetches models from Ollama `/api/tags`.
2. `_infer_capabilities` infers capabilities from name/families/capability flags.

3. `_base_score` combines size/context/capability count.
4. `pick(task)` resolves by precedence:
 1. custom rule,
 2. preferred model setting,
 3. highest scored capable model,
 4. fallback to highest scored available model.

Task types in code: reasoning, coding, ocr, embedding, translation, summarization, vision, chat.

Snapshot output shape:

```
{
  "last_refresh": "2026-06-30T...",
  "last_error": null,
  "count": 5,
  "models": [
    {
      "model_name": "qwen3.5:4b",
      "capabilities": ["chat", "reasoning", "summarization"],
      "context_length": 32768,
      "score": 3.1
    }
  ],
  "avg_score": 2.7,
  "custom_rules": {"embedding": "qwen3-embedding:4b"}
}
```

1.3.6 3.6 Flow F: RAG ingestion and retrieval

Relevant files:

- router: `apps/api/src/ai_studio/api/routers/rag.py`
- service: `apps/api/src/ai_studio/services/rag_service.py`

Ingestion paths:

1. `POST /rag/documents` registers Document row (`status="registered"`).
2. `POST /rag/documents/{id}/ingest` uploads file and calls `ingest_local_file`.
3. `POST /rag/connectors/ingest` supports connector github or web.

Text extraction support in `_extract_text`:

- .pdf via `pypdf`,
- .html/.htm via `BeautifulSoup`,
- .docx via `python-docx`,
- .csv/.xlsx via `pandas`,
- fallback plain text read.

Indexing path (`_index_text`):

1. `_chunk_text` with overlap.
2. pick embedding model via `model_router.pick("embedding")`.

3. create Qdrant collection `studio_chunks` if needed.
4. upsert vectors and payload metadata.
5. persist `EmbeddingMetadata` rows and set document status="ingested".

Retrieval path (retrieve):

1. Build owner filter + optional metadata filters.
2. Semantic query in Qdrant when mode is semantic or hybrid.
3. Keyword candidate scoring when mode is keyword or hybrid.
4. Normalize and combine scores.
5. Optional rerank with cosine similarity using embedding model.
6. Return `RetrievalResponse` with hits, highlights, and metadata.

Retrieval request shape:

```
{
  "query": "Which GPU and VRAM does this platform target?",
  "top_k": 5,
  "mode": "hybrid",
  "rerank": true,
  "candidate_pool": 30,
  "filters": {}
}
```

Retrieval hit shape:

```
{
  "document_id": "...",
  "chunk_id": "...",
  "score": 0.912341,
  "text": "...",
  "highlights": ["..."],
  "metadata": {
    "chunk_index": 1,
    "source_uri": "...",
    "name": "...",
    "semantic_score": 0.88,
    "keyword_score": 0.27,
    "embedding_model": "qwen3-embedding:4b"
  }
}
```

1.3.7 3.7 Flow G: Tool execution with role + confirmation + audit

Relevant files:

- `apps/api/src/ai_studio/api/routers/tools.py`
- `apps/api/src/ai_studio/tools/filesystem.py`
- `apps/api/src/ai_studio/tools/terminal.py`
- `apps/api/src/ai_studio/tools/python_exec.py`

Security mechanics:

1. `_ensure_tool_role` enforces RBAC per tool name.
2. `_validate_arguments` validates argument types against tool manifest schema.
3. `require_confirmation_token` requires `X-Confirm-Token: CONFIRM-<APP_ENV_UPPER>` for destructive tools.
4. `append_audit_log` records execution metadata.

Filesystem constraints:

- all paths must remain under `AI_STUDIO_ALLOWED_ROOT` (default `/home/ahmad/AI`, fallback `/app`).

Terminal/Python execution mode:

- if `TOOL_EXECUTION_MODE=container_preferred` and Docker is available, run in sandbox container first;
- fallback to host execution if container run fails (except timeout handling).

Tool execute request shape:

```
{
  "name": "filesystem.search",
  "arguments": {
    "root": "/home/ahmad/AI/multi-agent-ai-studio/apps/api/src",
    "pattern": "class Settings",
    "file_glob": "**/*.py",
    "max_results": 5
  }
}
```

1.3.8 3.8 Flow H: Async run queue flow (/runs + ARQ worker)

Relevant files:

- API: `apps/api/src/ai_studio/api/routers/runs.py`
- worker: `workers/arq/src/ai_studio_worker/main.py`

Step-by-step:

1. `POST /runs` creates `AgentRun(status=queued)`.
2. Enqueues job `execute_agent_run` with run ID into Redis queue.
3. Worker dequeues run ID.
4. Worker creates Ollama + router + runtime.
5. Worker runs `runtime.run_with_trace(prompt)`.
6. Persists output/model_usage/status and creates `TraceRecord`.

Run create request (`RunCreate`):

```
{
  "workflow_id": "optional-workflow-id",
  "agent_id": "optional-agent-id",
  "input_payload": {"prompt": "..."}
}
```

1.3.9 3.9 Flow I: Telemetry and system monitoring

Relevant files:

- collector loop: `apps/api/src/ai_studio/state.py`
- snapshot service: `apps/api/src/ai_studio/services/system_metrics.py`
- GPU monitor: `apps/api/src/ai_studio/services/gpu_monitor.py`
- API: `apps/api/src/ai_studio/api/routers/system.py`

How it works:

1. Background task `_collect_system_metrics_loop` runs every `METRICS_SAMPLE_INTERVAL_SECONDS`.
2. Uses `psutil` for CPU/RAM and `nvidia-smi` via subprocess for GPU stats.
3. Persists `SystemMetricSample` row to PostgreSQL.
4. GET `/system/metrics/timeseries` returns sampled metrics for dashboard/system pages.

Timeseries response item keys:

- `recorded_at`,
- `cpu_percent`,
- `memory_total_mb`, `memory_used_mb`, `memory_percent`,
- `gpu_available`, `gpu_name`, `gpu_total_mb`, `gpu_used_mb`, `gpu_free_mb`,
- `gpu_utilization_percent`, `gpu_memory_utilization_percent`,
- `metadata`.

1.4 Module 4: Setup & Run Guide

This section describes declared setup/run paths from repository files only. No commands were executed for this handbook run.

1.4.1 4.1 Prerequisites

1. Linux environment.
2. `uv` installed.
3. Python 3.12.10 available (Makefile and scripts target this).
4. Docker + Docker Compose.
5. Node.js for frontend (`apps/web/package.json` uses Next 15).
6. Ollama runtime available.

1.4.2 4.2 Local setup (uv path)

From repository root:

```
cd /home/ahmad/AI/multi-agent-ai-studio
cp .env.example .env
UV_CACHE_DIR=/home/ahmad/AI/.uv-cache uv venv --python 3.12.10
UV_CACHE_DIR=/home/ahmad/AI/.uv-cache uv sync --group dev
PYTHONPATH=apps/api/src UV_CACHE_DIR=/home/ahmad/AI/.uv-cache uv run alembic -c apps/api/al
```

Equivalent via Makefile:

```
make setup
```

1.4.3 4.3 Run backend, worker, frontend (non-Docker)

Backend API:

```
make run-api
```

Worker:

```
make run-worker
```

Frontend:

```
cd apps/web
npm install
npm run dev
```

1.4.4 4.4 Run full stack with Docker Compose

```
docker compose up -d --build
```

Core services started by default:

- api, worker, ollama, web, postgres, redis, qdrant, minio, otel-collector.

Optional profile:

- observability adds clickhouse and langfuse-web.

Examples:

```
docker compose --profile observability up -d --build
```

1.4.5 4.5 Required environment keys and what they control

Application/auth:

- APP_ENV, APP_HOST, APP_PORT, APP_SECRET_KEY, APP_JWT_ALGORITHM, APP_ACCESS_TOKEN_MINUTES, APP_REFRESH_TOKEN_MINUTES, APP_ALLOWED_ORIGINS.

API/web ports:

- API_HOST_PORT, WEB_HOST_PORT, NEXT_PUBLIC_API_BASE_URL.

PostgreSQL:

- POSTGRES_HOST, POSTGRES_PORT, POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_HOST_PORT.

Redis/Qdrant/MinIO:

- REDIS_URL, REDIS_HOST_PORT, QDRANT_URL, QDRANT_HOST_PORT, MINIO_ENDPOINT, MINIO_ACCESS_KEY, MINIO_SECRET_KEY, MINIO_BUCKET, MINIO_API_HOST_PORT, MINIO_CONSOLE_HOST_PORT.

Ollama/model router:

- OLLAMA_BASE_URL, OLLAMA_HOST_PORT, OLLAMA_MODELS_PATH, OLLAMA_REQUEST_TIMEOUT, OLLAMA_HEALTH_TIMEOUT, OLLAMA_BOOTSTRAP_MODELS, OLLAMA_AUTO_PULL_MODELS, OLLAMA_PREFERRED_MODEL, OLLAMA_PREFERRED_EMBEDDING_MODEL, OLLAMA_DISABLE_THINKING, MODEL_ROUTER_REFRESH_SECONDS.

Telemetry/observability:

- METRICS_SAMPLE_INTERVAL_SECONDS, OTEL_EXPORTER_OTLP_ENDPOINT, LANGFUSE_HOST, LANGFUSE_PUBLIC_KEY, LANGFUSE_SECRET_KEY, OTEL_HOST_PORT, LANGFUSE_HOST_PORT, CLICKHOUSE_HTTP_HOST_PORT.

OAuth:

- GITHUB_OAUTH_CLIENT_ID, GITHUB_OAUTH_CLIENT_SECRET, GITHUB_OAUTH_CALLBACK.

Tool sandbox/security:

- RATE_LIMIT_RPM, TOOL_SANDBOX_IMAGE, TOOL_EXECUTION_MODE, TOOL_CONTAINER_NETWORK, TOOL_CONTAINER_CPUS, TOOL_CONTAINER_MEMORY.

Filesystem-specific runtime key used by tools/rag:

- AI_STUDIO_ALLOWED_ROOT for tool root restriction.
- AI_STUDIO_UPLOAD_DIR for RAG upload temp storage.

1.4.6 4.6 Database migration and initialization facts

1. Migration entrypoint: apps/api/alembic.ini + apps/api/alembic/env.py.
2. Main migration files are 20260627_0001_initial.py (core tables) and 20260628_0002_system_metrics.py (system_metrics table).
3. Runtime startup enforces migration head match via ensure_schema_current.
4. No dedicated seed script is present; initial user typically created via /auth/register.

1.4.7 4.7 Typical command sequence for a clean machine

```
cd /home/ahmad/AI/multi-agent-ai-studio
cp .env.example .env
make setup
make run-api
make run-worker
cd apps/web && npm install && npm run dev
```

Or containerized:

```
cd /home/ahmad/AI/multi-agent-ai-studio
docker compose up -d --build
```

1.5 Module 5: Study Plan & Practice Exercises

1.5.1 5.1 Recommended study order for new learners

1. Read README.md, pyproject.toml, .env.example, docker-compose.yml, Makefile.
2. Read backend bootstrap path: main.py, state.py, core/config.py, db/session.py.
3. Read domain model: models/entities.py and Alembic versions.
4. Read auth + dependency guards: api/deps.py, api/routers/auth.py, core/security.py.
5. Read orchestration core: services/agent_runtime.py, services/model_router.py, services/ollama.py.
6. Read chat/runs/traces routes: api/routers/chat.py, runs.py, traces.py, worker ai_studio_worker.py.
7. Read RAG path: api/routers/rag.py, services/rag_service.py.
8. Read tools path: api/routers/tools.py, tools/filesystem.py, tools/terminal.py, tools/python_exec.py.
9. Read frontend integration: apps/web/lib/api.ts, components/chat/chat-panel.tsx, components/workflow/editor.tsx, then pages.
10. Read tests in apps/api/tests to reinforce expected behavior.

1.5.2 5.2 Practice exercises

1. Trace the end-to-end sync chat path. Question: starting from `POST /api/v1/chat/{chat_id}/messages`, list each DB table that is written and in what order.
2. Explain routing logic. Question: for prompt text "OCR Agent Task: extract table from scan", what `task_type` is selected and which capability is requested from `ModelRouter`?
3. Validate tool safety model. Question: identify all conditions required to run `POST /api/v1/tools/terminal` successfully.
4. Reconstruct retrieval ranking. Question: explain how `RagService.retrieve` combines semantic and keyword signals in hybrid mode before optional reranking.
5. Inspect workflow validation. Question: list three concrete validation/warning rules implemented by `WorkflowCompiler.compile`.
6. Session refresh mechanics. Question: from `apps/web/lib/api.ts`, describe what happens when an API call gets HTTP 401.
7. Background queue flow. Question: what exactly does `POST /api/v1/runs` do immediately, and what is deferred to worker execution?
8. Metrics persistence path. Question: which function writes rows to `system_metrics`, and which endpoint reads them back?

1.5.3 5.3 Solution outlines

1. Sync chat writes: `agent_runs` (queued/running/completed), `chat_messages` (user then assistant), `traces`, and `audit_logs` for completion/failure action.
2. Routing result: `_route_task_type` detects ocr via keyword; `_specialist` maps ocr to capability "ocr"; router asks `ModelRouter.pick("ocr")`.
3. Terminal execution conditions: valid bearer token, role owner (via `owner_access_guard()`), confirmation header `X-Confirm-Token: CONFIRM-<APP_ENV_UPPER>`, passing rate limit, safe command (not blocked patterns), cwd under allowed root.
4. Hybrid retrieval scoring: semantic candidates from Qdrant vector search + keyword candidates from scroll + term scoring; normalize both; combined score $0.7 * \text{semantic} + 0.3 * \text{keyword}$; optional rerank blends current score and cosine similarity with query embedding.
5. Workflow compiler rules examples: non-empty node list required; duplicate node IDs rejected; entrypoint must exist; edges cannot reference unknown nodes; self-edge allowed only for loop; warnings for unreachable nodes and malformed conditional edges.
6. 401 refresh behavior: `apiRequest` calls `refreshAccessToken()` once (except when already on `/auth/refresh`); if refresh succeeds, retries original request with new token; if refresh fails, clears local auth state.
7. Runs endpoint split: API creates queued `AgentRun` and enqueues `execute_agent_run`; worker later runs supervisor pipeline, updates run output/status, writes trace.
8. Metrics path: writer is `_collect_system_metrics_loop` in `state.py`; reader is `GET /api/v1/system/metrics` in `system.py`.

1.6 Understanding Checklist

Use this checklist to confirm mastery:

1. Can you explain `main.py` -> `build_app_state` -> `router` handlers startup flow without looking at code?
2. Can you describe JWT access/refresh and Redis-backed refresh revocation used in `auth.py`?
3. Can you walk through `planner` -> `decomposer` -> `router` -> `specialist` -> `reviewer` -> `critic` with real function names?
4. Can you explain how `ModelRouter.pick` chooses a model for embedding vs ocr?
5. Can you explain what data gets persisted in `AgentRun`, `ChatMessage.token_usage`, and `TraceRecord.meta` during a chat run?
6. Can you describe how RAG ingestion differs for file uploads vs GitHub/web connectors?
7. Can you identify when and why a confirmation token is required for tools?
8. Can you map each major frontend page to its backend route group?
9. Can you explain the ARQ worker's role and what is asynchronous vs synchronous in this architecture?
10. Can you list the minimum env keys needed to boot local API + DB + Ollama + web path?